

ARCHITECTURAL BLUEPRINT

TWS Forecasting — Architectural Blueprint

A regime-aware, state-reconstruction pipeline for forecasting global Total Water Storage anomalies from GRACE/GRACE-FO gravimetry, built for the Zindi/ITU "A Step Ahead of Drought" competition.



CONTENTS

Table of Contents

01	Overview and competition context
02	Governance and the document set
03	Core architectural thesis
04	Information-set discipline
05	System architecture
06	Repository and module map
07	Data architecture and the leakage firewall
08	Masking architecture
09	Feature architecture
10	Model architecture: the champion ladder and kill criteria
11	Validation engine
12	Uncertainty architecture
13	Phase-by-phase architecture
14	Submission and candidate subsystem
15	Definition of champion
16	Assumptions and open questions
17	Explicit anti-goals

18 Reproducibility

19 The governing rule

20 Final architectural statement and current status

Section links jump to each section in this on-screen copy; in the exported PDF, use your reader's own page list alongside this contents page.

SECTION 01

Overview and competition context

The competition asks entrants to forecast next-month Total Water Storage anomalies, derived from the GRACE and GRACE-FO satellite gravimetry missions, at a fixed global grid of 15,715 land-only locations. The training data covers 2002 through part of 2015 (2,154,021 rows); the test set covers 18 non-contiguous months between 2016 and 2018 (280,961 rows) drawn from the same 15,715 locations, meaning this is temporal extrapolation at known locations rather than spatial extrapolation to new ones.

The prize pool is €2,000: €1,000 for first place, €600 for second, €400 for third, plus Zindi ranking points. The competition opened 9 July 2026 and closes 13 September 2026, with the private leaderboard revealed the same day. Scoring is split evenly: 50% is Competition Phase 1, the leaderboard RMSE on the public split; 50% is Competition Phase 2, a written report scored on AI Trustworthiness (30% of the total) and Innovation & Practicality (20% of the total), judged only for the top 10 finishers. The public leaderboard reflects roughly 30% of the 280,961 test rows (~84,000 rows); the remaining 70% is the private leaderboard, revealed only at close and never visible during the competition — the number shown on the public leaderboard during the competition is not the number that decides placement.

Competition rules require an open-source solution with no AutoML; pretrained models are permitted if openly available. External data is permitted only if it is freely and operationally available within one month of acquisition and is fully documented — source, latency, and provenance. Random seeds must be fixed, because Zindi re-runs the top 10 finishers' code after close and adjusts final rank to whatever score that re-run actually reproduces, not the originally submitted score. The top 10 are given a 72-hour window to provide code, model, and report on request; the top 3 are required to make their solution public. Each entrant may select at most 200 submissions across the competition, at most 5 per day, and must nominate exactly 2 submissions for private-leaderboard judging before the competition closes — by default the platform uses the entrant's best 2 public-leaderboard submissions, but a different pair can be nominated deliberately.

Three personal goals sit behind this system, in priority order: win the competition outright; build genuine, transferable data science and machine learning skill in the process; and produce a project that stands on its own as evidence of senior-level data science and machine learning engineering capability, independent of whether it wins. The architecture in this document is built to serve all three simultaneously — nothing here is scoped down to "just enough to win" at the expense of being a defensible, well-engineered system, and nothing here is scoped up into unearned complexity that the competition timeline and two-person team size cannot actually support.

SECTION 02

Governance and the document set

Four kinds of document exist for this project, each with a distinct job, and together they form a strict precedence order for resolving any conflict:

1. **The competition's own rules**, published by Zindi, take precedence over everything below without exception. Nothing in this system may violate them, regardless of what any other document says.
2. **This document, the Architectural Blueprint**, specifies what the system is: its components, their responsibilities, the contracts between them, and the invariants that must hold. It is the second-highest authority.
3. **The Project Plan** sequences the work into an ordered set of phases, each with a concrete Definition of Done, and tracks execution status. It answers "in what order, and how do we know a phase is finished," not "what is the system."
4. **The Competitive Analysis** holds the strategic reasoning behind the architecture: the full catalogue of modeling hypotheses considered, their confidence and expected impact, the leaderboard context, and the reasoning chains that motivated architectural choices. It answers "why this approach and not another," in more exploratory and discursive form than belongs in an architecture specification.

Below all four sits the empirical record itself — experiments, their results, and the code that implements the system — which is where evidence actually originates.

Architectural Decision Records (ADRs) are the mechanism by which evidence is allowed to change any of the above without silent drift. When a finding contradicts something stated in this document, the response is never to quietly start doing something different: an ADR is written recording the context, the evidence, the decision, the alternatives considered, and the consequences; this document is updated to reflect the decision; the Project Plan is updated if the change affects sequencing; and only then does

implementation proceed. Code is never allowed to diverge silently from this document, and this document is never allowed to diverge silently from what an experiment actually showed. Two registers exist alongside the ADR log for material that is not yet a decision: an assumptions register, recording things currently believed and acted on, and an open-questions register, recording things currently unresolved and not expected to resolve soon. Both are described fully in §16.

SECTION 03

Core architectural thesis

The central design decision of this system is a reframing of the forecasting problem itself. The naive framing is direct one-step regression: predict $TWS(t+1)$ from $TWS(t)$, SPEI, and soil moisture. The architecture instead treats this as a **state-reconstruction and state-transition problem**: reconstruct the hydrological state of a known location from its full historical observation record and environmental context, then forecast how that state evolves one month forward.

This reframing is not a stylistic preference; it is forced by the structure of the data. Sixty-six percent of test rows have no current TWS observation at all — the value is masked, mirroring the real 2017–2018 observational gap between the GRACE and GRACE-FO satellite missions. This masking does not occur as scattered, independent missingness across rows; it arrives in contiguous whole-month blackout blocks in which upward of 99.7% of the entire 15,715-location grid is masked simultaneously, with only a handful of scattered, non-recurring unmasked rows per blackout month. A model that only performs well when the current TWS observation is present will fail on two-thirds of the test set by construction.

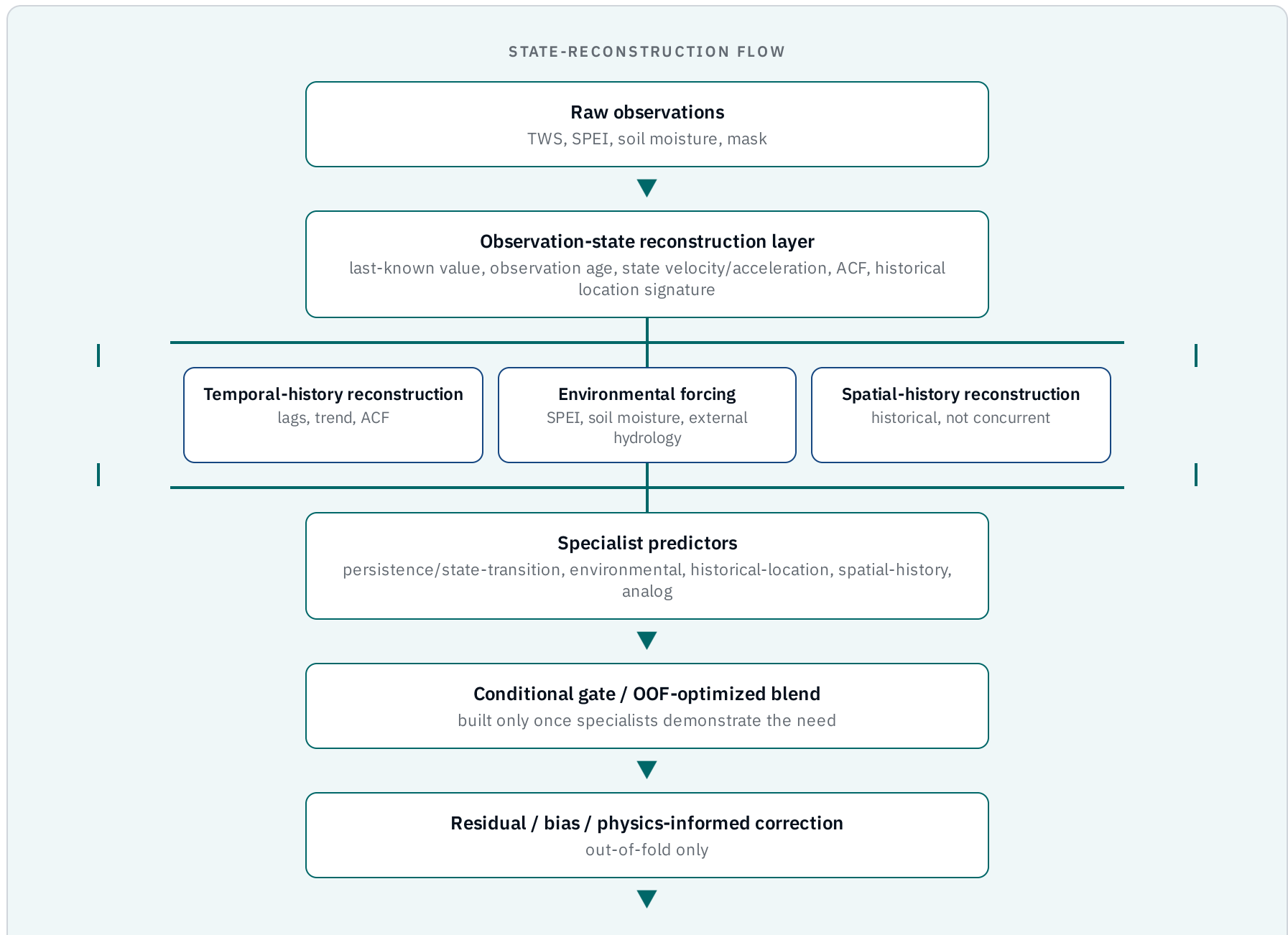
The evidence for this framing is direct and was measured, not assumed. A naive persistence predictor — assume no change from the last known value — achieves an in-sample RMSE of 0.572 against a target standard deviation of 0.912, meaning month-to-month autocorrelation in TWS is strong. A global mean predictor achieves only 0.912; per-location, per-calendar-month seasonal climatology achieves 0.817, weaker than intuition would suggest, meaning most of the error in this problem comes from within-location month-to-month deviation, not from misjudging a location's baseline level. Correlation structure collapses when the target is redefined as a delta rather than a level: TWS_t correlates 0.80 with the level of next month's value but only -0.32 with the *change* (dominated by mean-reversion), and the strongest available SPEI feature correlates only -0.065 with the delta — meaning "persistence plus a learned correction" has a low ceiling on the unmasked third of the data alone, and most of the room to improve on persistence lies specifically in the masked regime. Same-month spatial correlation between neighboring grid cells is very high

(0.981) but close to useless during a blackout, because the whole grid loses observation together — a masked cell's same-month neighbors are almost always masked too, so spatial signal in the regime that matters most has to come from neighbors' *historical* trajectories, not their concurrent readings.

One further empirical finding is treated as a hard gate on the rest of the system rather than a footnote: persistence RMSE by year is stable, in the 0.50–0.63 range, from 2002 through 2014, then jumps to 0.898 in 2015. The cause is not yet established — it may be a partial-year artifact, since the training data only covers January through August of 2015, or it may reflect a genuine data-quality or regime shift late in the GRACE mission's operational life, which is documented to have experienced sensor and battery degradation toward its end. Until this is resolved, the system does not finalize any trend, recency-weighting, or climatology feature design that depends on trusting the full 2002–2014 record equally, because if 2015 reflects a real regime change, it directly affects how much the 2002–2014 period should be trusted as a guide to the 2016–2018 test period.

The state-reconstruction architecture explains all of the above findings simultaneously: why persistence is unusually strong, why the delta is much harder to predict than the level, why two-thirds of test rows lack current TWS, why masking arrives in contiguous month-level blocks rather than scattered per-row, why per-location history matters, why staleness matters, why same-month spatial neighbors go dark together, and why external hydrological forcing data should matter more, not less, precisely when TWS itself is unavailable. The system's competitive advantage is not intended to come from a better gradient-boosted model in isolation — it is intended to come from a better representation of hydrological state when the most informative single observation is missing.

One caveat is stated explicitly because it affects how results are reported: "hydrological state" is this system's modeling hypothesis, not something directly observed. What is actually observed is TWS, SPEI, soil moisture, and a mask indicator; state is an inferred construct built from those four things, not a fifth measured quantity.



TWS(t+1)

SECTION 04

Information-set discipline

Every prediction the system produces is implicitly of the form $\hat{y}(t+1) = f(I_t)$, where I_t is everything knowable at the forecast origin t and strictly nothing else. This is a stricter requirement than simple timestamp ordering. A value labeled "January 2017" in an external reanalysis dataset is not automatically something a forecast made in January 2017 could actually have used, because reanalysis and satellite-derived products are frequently revised and reprocessed after their initial release — using a later-revised value would be a subtle but real form of leakage even though the timestamp looks correct.

Two schemas make this discipline concrete and enforceable in code rather than aspirational.

ForecastOrigin

Defined once per training, validation, or prediction example, with fields `origin_time`, `target_time`, `horizon`, `information_cutoff`, `location_id`, `regime`. It is what allows the system to state precisely that a given prediction targets September 2015 using only information available as of August 2015, rather than the vaguer notion of "a 2015 row." It is the join key that every other artifact in the system — state snapshots, features, out-of-fold predictions — keys off.

StateSnapshot

The single, canonical definition of what is known about a location's water state at a given forecast origin, computed once by the state-reconstruction module and consumed identically by every downstream feature, model, and diagnostic — no other module is permitted to compute its own competing notion of "months since observation" or "last known value." Its fields are `last_known_tws`, `last_known_time`, `months_since_observation`, `previous_known_tws`, `historical_delta`, `local_trend`, `seasonal_position`, `acf_1_3_6_12`, `observation_density`, `blackout_streak_length`, `location_signature`, `state_status`, where `state_status` takes one of three values — `OBSERVED`, `RECONSTRUCTED`, or `PARTIALLY_RECONSTRUCTED` — and this single

field is what allows the mixture-of-experts gating logic, the uncertainty architecture, the explainability reporting, and the deployment application's user interface to all answer the same underlying question, "how much do we actually know about this location right now," through one shared mechanism rather than four independently-implemented approximations of it.

A related invariant governs anything computed per-location from history — most importantly location signatures (mean, standard deviation, trend, seasonality amplitude, and autocorrelation at multiple lags, computed per location). It is not sufficient for a signature to be computed "out of fold" in the ordinary cross-validation sense, because a signature built from the entire 2002–2015 record and merely excluded from one particular fold still uses information from after the forecast origin in every other fold. The correct guarantee is that a signature used for a forecast originating at time `t` must be `signature(location, t)`, built only from history strictly before `t` — never `signature(location)` built from the full record. Fold-level out-of-fold computation and origin-time-indexed computation are different guarantees, and only the second is actually correct for this system. This is a checked architectural invariant: the leakage-testing module verifies it mechanically rather than relying on it being implemented correctly by convention.

SECTION 05

System architecture

The system is organized into nine functional layers plus a governance spine that runs alongside all of them rather than sitting above or below.

1. Data acquisition — train/test/external, dataset manifest
2. Data contract & versioning — schema, hashes, invariants
3. Forecast information set — origin/cutoff enforcement, external-data vintage lock
4. State reconstruction — StateSnapshot, observed/reconstructed status, signatures, spatial history
5. Feature engine — temporal / environmental / spatial-history / external / target transforms
6. Validation engine — 3-tier CV, masking scenarios, leakage firewall, decomposition table, degradation slope
7. Model research — baselines → GBM → specialists → MoE → OOF ensemble → residual/bias → physics-informed correction
8. Evidence & candidate system — OOF store, candidate registry, champion/challenger, uncertainty
9. Competition / production — submission subsystem serving

GOVERNANCE SPINE

Git · MLflow · ADRs · Assumptions · configs · seeds

Two further diagrams describe the same system from different angles, because they answer different questions than the layer stack above does.

The research-evidence loop

Describes how any single finding becomes a decision: a hypothesis leads to an experiment, run under the validation engine; the experiment produces evidence in the form of the error-decomposition table and out-of-fold predictions; that evidence supports a decision; the decision produces a candidate; the candidate may become a submission; and any new evidence that submission generates feeds back into the next hypothesis.

The lineage chain

Describes how any single prediction traces backward to its origin: a submission traces to a candidate, which traces to a specific MLflow run, which references a specific model version, feature version, and validation version, which was produced from a specific git commit operating on a specific, hash-verified dataset manifest. Every submitted prediction can be walked back through this chain to the exact code, configuration, and data that produced it.

SECTION 06

Repository and module map

```

twS-forecast/
|-- data/{raw,interim,processed,external}/      manifest-hash verified, raw gitignored
|-- src/tws_forecast/
|   |-- data/                                  loaders.py, contracts.py (pandera), versioning.py
|   |-- state/                                reconstruction.py (StateSnapshot, ForecastOrigin), signatures.py
|   |                                           (time-indexed + shrinkage), spatial_history.py
|   |-- features/                             temporal.py, environmental.py, external.py (vintage lock),
|   |                                           targets.py, masking.py
|   |-- validation/                           splitters.py, masking_simulator.py (MaskingScenario objects),
|   |                                           scenarios.py (registry, config-driven), tiers.py,
|   |                                           decomposition.py, leakage_tests.py
|   |-- models/                               baselines.py, gbm.py, specialists.py, moe.py, ensemble.py,
|   |                                           correction.py, research/deep.py (sandboxed – no core
|   |                                           pipeline dependency on it unless it wins)
|   |-- pipelines/                            train.py, evaluate.py, predict.py, submit.py, deploy.py
|   |-- serving/                              api.py, schemas.py, app.py
|   |-- utils/                                config.py (pydantic), logging.py, seeds.py, reproducibility.py
|-- artifacts/oof/<candidate_id>/              OOF prediction store – first-class, not notebook arrays
|-- submissions/                              submission_log.csv, candidates.csv, files/, manifests/
|-- configs/                                  base.yaml, validation/ (scenario configs), features/, models/, champion.yaml
|-- notebooks/, tests/ (incl. tests/data/golden/ – small fixed dataset for regression tests)
|-- docs/                                     PROJECT_PLAN.md, COMPETITIVE_ANALYSIS.md, ARCHITECTURE.md,
|                                           ASSUMPTIONS.md, OPEN_QUESTIONS.md, DATA_DICTIONARY.md,
|                                           MODEL_SELECTION.md, BIAS_AND_EXPLAINABILITY.md, MODEL_CARD.md,
|                                           CASE_STUDY.md, SETUP.md, adr/
|-- reports/{figures,experiments,submissions}/, models/, docker/, .github/workflows/
|-- mlruns/, mlflow.db (live from the project's first working phase, not deferred)
|-- requirements.txt (pinned), requirements-dev.txt, environment.yml, README.md, docker-compose.yml

```

SECTION 07

Data architecture and the leakage firewall

The dataset consists of a training file, a test file, and a sample submission file, all sharing the same fixed grid of 15,715 land-only locations. The training file contains 2,154,021 rows spanning January 2002 through August 2015; the test file contains 280,961 rows spanning 18 non-contiguous months between 2016 and 2018. The grid is land-only and heavily Northern-Hemisphere-weighted — 5,573 locations sit between 30° and 60° North, against 570 between 30° and 60° South — which is a genuine generalization and bias consideration for the system, not merely a descriptive footnote, and is treated as such in the bias and explainability work described in §13.

The target is next-month TWS, verified programmatically against the raw data with no ambiguity. A schema contract, enforced with `pandera`, organizes the data's invariants into four groups: **temporal** (valid, monotonically ordered dates at monthly frequency), **spatial** (the fixed 15,715-cell grid, with no locations appearing or disappearing between train and test), **target** (the target column is always exactly next-month TWS, never silently redefined), and **masking** (a row's TWS is masked if and only if its mask indicator says so — masking status is never silently altered by a transform).

The leakage firewall

The leakage firewall enumerates, explicitly, what information the system is and is not permitted to use:

Forbidden	Allowed
Future TWS values	Test rows' own covariates
Target-derived test information	Historical observations, any depth
Future-released or reprocessed external covariates that were not actually available at the forecast origin	Correctly-vintaged external data
Signatures or climatology computed using rows at or after the forecast origin	Historical spatial or temporal aggregates, computed strictly before the forecast origin
Exploitation of the public/private leaderboard split	Model ensembles, out-of-fold meta-features
Manual editing of a generated prediction file	Config-driven, reproducible pipelines
Selecting features because they moved the public leaderboard score	Moderate public-leaderboard sanity-checking of an otherwise CV-driven decision

This firewall is checked mechanically, not only documented: a future-row shuffle test confirms that shuffling data after the forecast origin does not change a prediction made before it; a historical-only check confirms signatures and climatology features never draw on rows at or after their forecast origin; rolling-window features are verified to stop exactly at the information cutoff; and the masking simulator is checked to confirm it cannot leak a value it has just hidden.

Two further pieces of infrastructure support reproducibility of the data itself. Rather than adopting a dedicated data-versioning tool, the system keeps the raw competition files gitignored under `data/`, documents them in a tracked `data/README.md`, and pins their exact identity in a tracked `dataset_manifest.json` containing a SHA-256 hash and row count for each file — sufficient for a static, externally-supplied, non-redistributed dataset, and revisited only if multiple genuine dataset versions actually emerge.

Separately, because the competition explicitly requires documented packages and environment for any solution selected for review, the system maintains both a pinned `requirements.txt` describing the intended dependency versions and the environment's actual resolved state, so that "this environment is exactly reconstructable from scratch" is a checkable claim rather than an assumption.

SECTION 08

Masking architecture

The real structure of the test set's missingness has been reproduced and confirmed directly: blackout months mask upward of 99.7% of the entire grid simultaneously, with a handful of scattered, non-recurring exceptions — never a flat, row-independent probability of roughly 66%, and never a fixed set of "reference stations" that stay reliably observed. The 4 to 65 unmasked rows found within a given blackout month do not recur across different blackout months, with at most 0 to 2 rows of overlap between any pair checked — the exceptions are genuinely sporadic, not a calibration set that any feature should be built to rely on.

The masking simulator used throughout validation and training-time augmentation is driven by a configuration object rather than a bare masking rate. Its fields are `blackout_start`, `blackout_end`, `affected_locations`, `exception_rate`, `streak_length`, `transition_pattern`, `source/rationale` — this is what "streak-aware" means concretely, as an enforceable configuration contract rather than a design intention that individual code paths might or might not honor.

Two distinct uses of masking are never conflated with each other. *Historical* masking describes what actually happened to the real test months, and is used to characterize the true structure of the problem. *Synthetic* masking is artificially applied to historical periods where TWS is in fact known, in order to manufacture training and validation examples that resemble the real blackout structure. Treating the test set's own observed mask pattern as if it were itself a reusable training feature would mean training on the very thing the system is being evaluated against, which is not permitted.

SECTION 09

Feature architecture

Spatial features carry an explicit, declared information regime rather than being generated as one undifferentiated pool. Four categories are distinguished:

- **S1, concurrent** features (same-month neighbor TWS), which are valid only when TWS is actually observed and are therefore rarely useful in precisely the regime where they would matter most.
- **S2, historical** features (neighbor trajectories, trends, and anomalies), which are the primary mechanism available during blackout regimes.
- **S3, signature** features (static basin- or location-level characteristics).
- **S4, residual** features (spatial correction applied after a base model's prediction).

Every spatial feature in the system declares which of these four regimes it belongs to, which prevents a specific, previously-encountered failure mode: a concurrent-neighbor feature silently contributing nothing in exactly the regime it was built to help with, without that failure being visible anywhere in the feature's definition.

Any external data source is subject to a vintage lock: once a specific source and vintage is approved and pinned, it is never silently swapped for a newer or reprocessed version of the same nominal data without a new decision record. Approval proceeds through an ordered sequence of checks — scientific usefulness, temporal availability at the correct forecast origin, revision risk (does the source get reprocessed after initial release, and if so, would the version used actually have been available in near-real-time form), competition-rule compliance, operational accessibility, and complete documentation — and a source that fails any one of these checks is rejected before it reaches feature generation.

Knowledge of the real GRACE-to-GRACE-FO mission calendar is treated as data-generating-process metadata by default, not as a model feature by default. It is used to understand the masking mechanism and to design realistic synthetic blackout scenarios for validation. Turning something like "months since the GRACE mission gap began" into an actual model input is a separate decision, requiring its own experiment and its own decision record, because used carelessly it risks the model learning to identify which specific competition test month it is looking at, rather than learning a genuinely generalizable relationship.

Target framing itself is treated as a first-class experimental question rather than a settled baseline footnote. Predicting the raw level of $TWS(t+1)$, predicting a delta relative to reconstructed state, predicting an anomaly relative to climatology, predicting a trend residual, and predicting a volatility-normalized delta all compete against each other under identical validation folds before any one of them is adopted as the system's target framing. The persistence-plus-delta baseline is the floor this comparison has to clear, not a preordained conclusion about which framing wins.

SECTION 10

Model architecture: the champion ladder and kill criteria

The system's model architecture is not a single fixed model, and not an unconstrained search — it is a staged, evidence-gated progression, in which each stage must beat the previous one on the appropriate validation tier before the next stage is attempted, and any stage that fails to help is dropped rather than kept for its own sake. The full ladder, in order:

1. Global mean predictor
2. Seasonal climatology
3. Current-TWS oracle persistence
4. Last-known-state persistence
5. Persistence with a learned delta correction
6. A raw-feature gradient-boosted model
7. A gradient-boosted model enriched with the state-reconstruction layer (staleness, trajectory, velocity, and acceleration)
8. The same model further enriched with temporal-history features (lags, autocorrelation, trend, recency weighting)
9. Further enriched with shrinkage-regularized historical location signatures
10. Further enriched with historical spatial-history features
11. Further enriched with provenance-gated external hydrological data
12. Named specialist models
13. A conditional or mixture-of-experts blend of those specialists, built only if the specialists demonstrate genuinely regime-conditional advantages
14. An out-of-fold-optimized ensemble across specialists and blends
15. Out-of-fold residual and bias correction

16. A physics-informed blend incorporating a water-balance estimate, if external data supporting one has been sourced

17. Last and explicitly optional, a deep spatiotemporal model, evaluated honestly against the gradient-boosted champion rather than adopted by default

Gradient-boosted trees — LightGBM as the primary candidate, with XGBoost and CatBoost as comparison points — are the system's default modeling family, chosen because they handle missing values natively, which fits a problem defined by structured missingness, and because for a tabular problem at this scale, representation of the input (the state-reconstruction and feature layers) matters more than model architecture sophistication. A deep spatiotemporal model — a GRU, ConvLSTM, or Transformer variant — remains on the ladder as a genuine, honestly-evaluated comparison rather than a default choice or a rejected idea: if a carefully engineered gradient-boosted pipeline built on top of the state-reconstruction architecture matches or beats it, that is itself a defensible, senior-level finding worth reporting, not a gap to be embarrassed about. Any deep-learning work is developed in a sandboxed module with no dependency from the core pipeline, so that the system's primary path never acquires a GPU-stack dependency unless a deep model actually clears the bar set by the gradient-boosted champion.

Progression up the ladder is governed by explicit stopping rules, decided in advance rather than after the fact, so that no research direction becomes an open-ended, unbounded investigation:

- The spatial-history branch is retained only if it improves blackout-tier cross-validated RMSE by at least approximately 0.002 across multiple blackout folds, and is dropped otherwise.
- The external-data branch is retained only if it improves masked-regime RMSE specifically, without materially degrading any other regime, and only after the feature in question clears the provenance and vintage gate.
- The mixture-of-experts branch is not started at all until named specialists first demonstrate measurable regime-conditional advantages over each other, and if they do not, a simple out-of-fold-optimized linear blend is used instead and considered sufficient.

- The deep-learning branch is stopped after one reasonable, reproducible head-to-head experiment if the gradient-boosted champion remains ahead, rather than iterated on in an attempt to catch up.
- Analog forecasting — finding historically similar seasonal and drought conditions at the same location and using what followed — is retained only as an ensemble member if it measurably improves the blend, never promoted to a primary model without clear evidence that it earns that position.

SECTION 11

Validation engine

Validation is organized around three tiers, each answering a distinct question rather than being three arbitrary sets of folds. **Tier 1**, the standard forecastability tier, asks whether the system can predict a future month under ordinary observation conditions, using time-respecting expanding-window splits rather than random k-fold, since a random split would leak future information into training given the data's non-stationary trend. **Tier 2**, the blackout tier, asks whether the system can forecast after losing the current TWS observation, and how gracefully accuracy degrades as staleness increases, using a streak-aware masking simulator that nulls out TWS and its dependent features in contiguous multi-month blocks per location, matching the real structure described in §8 rather than an independent per-row probability. **Tier 3**, the test-matched tier, asks whether the system can reproduce the exact observation, masking, and environmental conditions the real test months represent, and exists purely for diagnosis and robustness assessment — it may draw on the test set's own *structure* (its dates, its covariates, its missingness pattern) but never on test *outcomes* (targets, inferred private-split membership, or leaderboard feedback used to shape which historical episodes get selected as test-matched analogs). Final model selection is always made against Tier 1 and Tier 2, using predefined historical folds, never by repeated tuning against Tier 3 analogs — that boundary is the difference between legitimate diagnostic design and a soft form of indirect test-set tuning, and it is treated as a hard rule rather than a guideline.

A related rule governs the test set's row ordering directly: it is treated as descriptive metadata only. No feature derived from a row's position or index within the test file is permitted, and no inference about which rows belong to the public versus private split is drawn from row order, regardless of what patterns might appear to be present in it.

Error decomposition and degradation slope

Every model evaluated under this system produces a full error-decomposition table, not a single aggregate RMSE, broken out along these dimensions:

Breakdown	Slices reported
Observation regime	Overall, masked-regime, unmasked-regime
Staleness bucket	1–2 months, 3–4 months, 5-plus months since last observation
Geography	By hemisphere
Data regime	Extreme-TWS and rapid-change slices of the data

A degradation slope — the change in RMSE per additional month since the last observation — is tracked as a fourth standing metric alongside RMSE, because it measures how gracefully, not merely how accurately, the system handles staleness. A model is only promoted from one stage of the champion ladder to the next when it improves the relevant validation tier without materially degrading another important regime, judged from this full decomposition table — a model that improves the aggregate number while becoming fragile in one regime is not promoted on that basis alone.

Each validation scenario used across the project is assigned an identifier and lives as a configuration file — for example, a standard expanding-window scenario, a seven-month synthetic-blackout scenario, a scenario approximating 2015-like conditions, and a scenario approximating a broader drought regime — and experiments reference these scenario identifiers rather than re-describing the same split logic inline each time. One validation-version tag, incremented whenever the validation logic itself meaningfully changes, is sufficient bookkeeping at this project's scale; a separate framework-version and scenario-version hierarchy layered on top of that would be more overhead than the team size and timeline justify.

Where a mixture-of-experts or gating architecture is used, the gate receives exactly the same information set the individual experts receive at prediction time, with no privileged signal — for instance, the gate cannot see the true current TWS value when deciding how much weight to give the specialist responsible for the missing-TWS regime, unless that value is genuinely available in the regime being evaluated. This is stated as an explicit rule because it is an easy and subtle way for a mixture-of-experts architecture to leak information without the leak being obvious from the architecture diagram alone.

Ensemble and residual-correction methods follow a mechanical out-of-fold discipline rather than a stated principle left to convention: an ensemble meta-model is trained only on out-of-fold predictions from base models that were never trained on the row being predicted, and is itself evaluated on a further held-out split, because a meta-model fit and evaluated on the same out-of-fold predictions produces deceptively strong validation numbers that will not generalize. Residual correction follows the identical chain — base model, out-of-fold prediction, out-of-fold residual, a residual model fit on that residual, added back to produce a corrected prediction — never a residual model fit on in-sample training predictions.

Finally, promotion of any candidate to champion status requires a reported range, not a point estimate: mean cross-validated RMSE together with its variance across folds and, near a genuine promotion decision, across three or four different reasonable random seeds. An improvement of a few thousandths of RMSE that sits inside the observed noise band is not treated as a promotion case, and a model that only outperforms with one specific seed is not considered robust enough to promote.

SECTION 12

Uncertainty architecture

The system distinguishes three separate sources of uncertainty rather than blending them into one undifferentiated confidence number. **Model uncertainty** captures whether the system's specialist models or ensemble members disagree with each other on a given prediction. **State uncertainty** captures how reconstructed, as opposed to directly observed, the current `StateSnapshot` is for the location and month being predicted — this is a direct function of the `state_status` field described in §4. **Regime uncertainty** captures how far outside the training distribution a given input sits, in the sense of covariate shift, independent of whether the state itself happens to be well observed.

These three signals are combined into a single prediction confidence signal, but that combined signal is only reported as "confidence" after it has been validated to actually behave like confidence: predictions are bucketed into confidence deciles, and observed RMSE within each decile is checked to confirm it rises monotonically as reported confidence falls. If that monotonic relationship does not hold, the signal is reported honestly as an "uncertainty proxy" rather than as calibrated confidence. This distinction is not merely an internal quality check — it is a direct requirement of the Trustworthiness section of the competition's report, which explicitly evaluates whether reported confidence or transparency signals are genuinely meaningful rather than cosmetic.

SECTION 13

Phase-by-phase architecture

The system is built in seventeen phases, numbered 0 through 16, each producing a specific architectural component and a concrete, checkable Definition of Done. This numbering is the project's own internal work breakdown and is unrelated to the competition's own "Phase 1" and "Phase 2" scoring structure described in §1; where confusion is possible, this document refers to "Project Phase N" explicitly.

Project Phase 0 — minimal reproducible foundation

Establishes just enough engineering discipline to run reproducible experiments, deliberately without full production polish this early, so that engineering does not compete with modeling insight for time before the system's core hypotheses are even tested. This phase establishes the git repository and standard module layout described in §6; a pinned Python 3.11 dependency set; the dataset manifest and hash-verification scheme described in §7; and a lightweight, queryable experiment log — initially a flat table, upgraded to a full MLflow tracking backend once the validation harness in Project Phase 2 stabilizes, so that Project Phase 1's findings are captured in the same lineage chain as everything that follows rather than living outside it.

Project Phase 1 — forecastability and the data-generating process

Establishes, empirically, exactly what the system is being asked to forecast and how difficult each regime actually is, before any validation design or modeling code is finalized. An ordered sequence of experiments is run, each building on the last: reproducing the real masking process directly against the data (already confirmed, as described in §8); resolving the 2015 persistence-RMSE anomaly described in §3, treated as a hard gate rather than an open curiosity, because the trend, recency-weighting, and climatology design in later phases depends on knowing whether the 2002–2014 record can be trusted uniformly; building a stratified blackout-degradation curve — RMSE as a function of months since the last observation, cut by latitude band, season, per-location autocorrelation, and drought regime, rather than a single averaged curve that could hide two very different underlying

populations — which is the single highest-priority experiment in this phase, because it directly motivates and calibrates the historical-signature features built in Project Phase 4; establishing a last-known-state baseline distinct from climatology; testing whether per-location autocorrelation explains the shape of the degradation curve, which determines whether staleness alone or staleness interacted with location-level persistence is the correct state variable; measuring covariate shift between the training and test distributions for SPEI, soil moisture, calendar month, and masking rate; and researching the real, externally documented GRACE-to-GRACE-FO mission timeline, which grounds masking-related features in the actual mission calendar rather than in inference from the test set's own sparse masked months alone.

Project Phase 2 — validation harness

Builds the three-tier validation engine described in §11 on top of what Project Phase 1 establishes empirically, rather than on assumptions made in advance. This phase implements the expanding-window splitting logic, the streak-aware masking simulator, the three validation tiers and their integrity boundary, the standing error-decomposition table, and the degradation-slope metric, and migrates the flat experiment log from Project Phase 0 toward the full MLflow backend.

Project Phase 3 — state-aware baselines

Establishes four distinct baselines, deliberately kept separate rather than collapsed into one baseline with a fallback bolted on, because conflating them would obscure what is actually being measured: an oracle-persistence baseline computed only on rows where the current TWS is genuinely observed, answering how hard the unmasked problem is on its own; a last-observation-carried-forward baseline for masked rows, answering how far historical state reconstruction alone can go with zero learned correction; a seasonal-climatology baseline, already measured at 0.817; and a hybrid baseline that uses the current observation when available and falls back to the last known value otherwise, representing the realistic "no machine learning" reference point for the actual test structure. A global mean and a Ridge regression are included as further reference points. Every baseline is logged with the full error-decomposition table from §11, establishing the exact numbers, broken out by regime, that every subsequent model in the champion ladder must beat.

Project Phase 4 — state reconstruction and feature engineering

Builds the observation-state reconstruction layer as one coherent, explicit pipeline stage — the `StateSnapshot` schema described in §4 — rather than as scattered, ad hoc features computed independently by different modules. Four distinct temporal quantities are built and kept explicitly separate, because under the real gap structure they are not interchangeable: calendar lag (the value at exactly `t minus k`, which is itself frequently missing inside a blackout streak), last-observed lag (the value at the most recent month it was actually observed, however far back that is), observation age (the staleness itself), and observation trajectory (the sequence of the last two or three actually observed values, from which state velocity and acceleration are derived), together with observation count and density over trailing 12- and 24-month windows. Historical location-signature features — mean, standard deviation, trend, seasonality amplitude, and autocorrelation at lags 1, 3, 6, and 12, along with each location's typical response to SPEI and soil moisture — are computed strictly out of fold and origin-time-indexed as required by §4, and are shrunk toward the global estimate using an empirical-Bayes-style weighting that increases with the amount of location-level evidence available, rather than used as naive per-location statistics, which would overfit given only around 150 monthly observations per location. Spatial-history features are built exclusively from neighbors' historical trajectories, trends, anomalies, seasonal signatures, and autocorrelation, never from same-month neighbor values, consistent with the S1–S4 taxonomy in §9 and the concurrent-correlation-but-unusable finding in §3. Seasonal and trend features, SPEI differencing and drought-persistence run-length features, and soil-moisture trajectory features are added alongside these, and the target-transformation comparison described in §9 (level, delta, anomaly, trend-residual, volatility-normalized delta) is run as one controlled experiment. Every transform in this phase is implemented as a leakage-safe transformer, fit only on the training fold it is applied within.

Project Phase 5 — core gradient-boosted forecasting

Builds the primary gradient-boosted model described in §10 on top of the state-reconstruction and feature layers from Project Phase 4. Training-time masking augmentation reproduces the same streak-aware, contiguous-block process used in validation, rather than a simpler row-independent masking scheme, because optimizing against an easier synthetic problem than the real one would produce a model that looks stronger in training than it performs on the actual blackout structure. Feature and state

representation is prioritized ahead of hyperparameter tuning in this phase, on the reasoning that representation determines far more of the achievable accuracy here than incremental tree-splitting adjustments do. The resulting champion is selected against the full error-decomposition table from §11, not against overall RMSE alone, and resource and efficiency metrics — training time, inference time, and model size — are logged for every model from this phase onward, both because they serve the Innovation & Practicality and Sustainability themes of the competition report directly, and because they are the actual evidence needed later to justify not using a deep-learning model if the gradient-boosted approach wins on both accuracy and cost.

Project Phase 6 — external data and spatial-history forecasting

Applies the provenance-and-vintage gate from §9 to any external hydrological data considered for inclusion, and tests a specific, sharper hypothesis than simply adding an external source: that external forcing data's value should be concentrated in the masked regime specifically, since it is the best remaining signal about state transition when TWS itself is unavailable. This is tested directly, by comparing masked-regime RMSE with and without the external features, rather than assumed. Water-balance composites — precipitation, evapotranspiration, and their difference at 1-, 3-, 6-, and 12-month windows — are built if precipitation and evapotranspiration data are sourced, and the spatial-history features from Project Phase 4 are extended with any external spatial or static data, such as elevation, land cover, or climate zone, that clears the provenance gate.

Project Phase 7 — specialists, mixture-of-experts, and out-of-fold ensembling

Builds named specialist models first, with no gating network: a persistence and state-transition specialist, an environmental-forcing specialist, a historical-location specialist, and a spatial-history specialist, each measured independently against the full error-decomposition table before any decision is made about combining them. A conditional gate or mixture-of-experts architecture is built only if the specialists demonstrate genuine, regime-conditional advantages over each other — for instance, one specialist reliably winning when current TWS is known and a different one reliably winning during long blackouts. If they do not show this pattern, a simple out-of-fold-optimized linear blend is used instead and is considered architecturally sufficient, because

the competition rewards RMSE, not architectural sophistication for its own sake. Analog forecasting — locating historically similar SPEI, soil-moisture, and seasonal conditions at the same location and using what followed as a secondary signal — is evaluated as a possible ensemble member under the same standard. The kill criteria described in §10 apply throughout this phase.

Project Phase 8 — residual, bias, and physics-informed correction

All correction in this phase is strictly out-of-fold, never fit on in-sample residuals. A second model is trained on the champion's out-of-fold residuals, using location, season, SPEI, months since the last TWS observation, and the original prediction as inputs, and its output is added back to the champion's prediction. Systematic bias by location, region, and season is measured and corrected, which doubles directly as evidence for the bias analysis required in Project Phase 9, since the competition's Trustworthiness rubric explicitly asks about bias. If Project Phase 6 sourced water-balance data, a physics-informed blend combines the machine-learned prediction with a physically estimated change in TWS, approximated as precipitation minus evapotranspiration, at a weight learned out-of-fold — a genuinely strong physics-guided-machine-learning narrative for the Innovation & Practicality theme, contingent on that external data having actually been sourced. Temporal and spatial smoothing of implausible prediction jumps is tested rather than assumed beneficial, since RMSE does not automatically reward smoothing and can penalize it if genuine extremes are washed out.

Project Phase 9 — trustworthiness, uncertainty, and explainability

Feeds the competition's Phase 2 report directly, which is scored 30% on Trustworthiness and 20% on Innovation & Practicality for the top 10 finishers. This phase is broader than SHAP analysis alone, since SHAP is useful evidence but is not itself synonymous with explainability for this problem. Global and regional SHAP analysis is produced, but as one input among several rather than the centerpiece. The model's stability is assessed — whether it behaves consistently across time, latitude, drought regime, and blackout duration. Its sensitivity is assessed — how predictions respond to small changes in SPEI, last-known TWS, or staleness. Its systematic failure modes are identified and explained. Bias is assessed by hemisphere, by wet versus dry regime, by high versus low TWS, and by short versus long blackout duration, drawing directly on the bias-correction analysis from Project Phase 8. The three-part uncertainty architecture described in §12 is built and calibration-checked here, producing the finding — genuinely

compelling and independent of whether the competition scores uncertainty directly — that the system's confidence signal narrows for fresh observations and widens for stale ones, meaning the model can be shown to know when its own state estimate is degrading. All of this is written up directly against the rubric's themes of bias, transparency, reusability, sustainability, efficiency, and practicality.

Project Phase 10 — production pipeline engineering

Promotes all notebook-stage logic into tested source modules covering data loading, feature computation, model wrapping, and pipeline orchestration, behind a single config-driven entrypoint. Test coverage is built for data loaders, feature transforms including the state-reconstruction layer, the model wrapper, and prediction consistency, alongside structured logging and full deterministic seed control throughout the pipeline.

Project Phase 11 — MLflow model registry and packaging

Upgrades the experiment-tracking backend from Project Phase 0's flat log to a full MLflow Model Registry with staging and production promotion stages, and packages the champion model as an MLflow `pyfunc` model that bundles the entire preprocessing and state-reconstruction pipeline, so that loading the registered model and calling predict on a raw input dataframe requires no separate manual preprocessing step. A model card is written documenting the registered model's intended use, training data, performance, and limitations.

Project Phase 12 — continuous integration and deployment

Establishes GitHub Actions workflows that lint and test every pull request and build and smoke-test the Docker image on every merge to the main branch, with the same pre-commit hooks used locally mirrored exactly in the CI environment, so that a deliberately broken pull request is blocked and a clean one passes end to end without manual intervention.

Project Phase 13 — deployment as an interactive application

Delivers a fully working, containerized application that runs locally with a single command and can be deployed publicly within roughly ten minutes to a free hosting target. A FastAPI backend exposes single and batch prediction endpoints and a health check. An interactive frontend lets a user select a location and month, see the state-reconstruction inputs that produced the prediction — last-known TWS, staleness, SPEI, soil moisture — and see the predicted next-month TWS together with an explanation and the uncertainty signal from Project Phase 9, alongside a world-map view of the grid. The deployed application always loads a specific, pinned model version from configuration rather than "whatever is latest," so that an unrelated experiment elsewhere in the project can never silently change what the live application serves. The application's interface surfaces model provenance directly — candidate identifier, validation version, and the state regime and staleness behind the current prediction. Input guardrails reject invalid coordinates, warn on covariates that fall outside the training distribution, and fall back gracefully through a validated hierarchy — full model, then a reduced-feature model, then state persistence, then climatology — if state reconstruction fails for a given input, rather than failing silently or returning a nonsensical prediction. Basic API hygiene is implemented as standard practice: request validation, payload size limits, timeouts, structured error handling, and logging, not because this is a high-risk application, but because calling something "production-ready" should mean something concrete and checkable.

Project Phase 14 — documentation and portfolio packaging

Produces a final `README.md` describing the problem, the approach, the architecture, a results table, how to run the system, and a link to the live demo. A case-study document tells the project's actual narrative arc explicitly: not "a gradient-boosted model was trained on GRACE data," but the genuine sequence of discovery — that the core challenge was not forecasting TWS directly but reconstructing hydrological state during systematic GRACE observation blackouts, followed by the masking-process discovery, the forecastability analysis, the state-reconstruction architecture, regime-aware validation, conditional forecasting, external hydrological forcing, and out-of-fold ensembling. That sequence is a stronger demonstration of statistical reasoning, missing-data modeling, competition strategy, leakage prevention, feature engineering, experiment design, and machine-learning engineering practice than the final model's score is by itself, and is written up as such.

Project Phase 15 — competition submission and the Phase 2 report

Final test predictions are validated exactly against the sample submission schema before upload. The final two submissions nominated for private-leaderboard judging are chosen using the error-correlation logic described in §14, not simply the two lowest cross-validated RMSE candidates. The competition's written report is produced against the real Trustworthiness Evaluation rubric: a short section on data and model bias, covering biases identified in the dataset (regional, environmental, or temporal) and any techniques used to assess or mitigate them; a section on model transparency, covering interpretability findings and which features were most influential; a section on the approach's reusability to other forecasting contexts and the design choices made to support that; and a section on sustainability and efficiency, covering the computational cost of training and any optimizations made, drawing on the resource metrics logged from Project Phase 5 onward.

Project Phase 16 — final quality assurance and retrospective

A full dry run is executed end to end on a clean checkout — from raw data, through training, to a deployed prediction — to confirm the system is genuinely reproducible from nothing but the repository and the documented setup instructions. A retrospective is written recording what was learned, serving as a genuine artifact of the project's process and as material for future discussion of the work.

SECTION 14

Submission and candidate subsystem

A candidate and a submission are treated as distinct concepts, because one candidate — one trained model and configuration combination — can produce several submissions over time as it is rerun with different seeds or minor tweaks. A candidate registry tracks each candidate's identifier, model family, model version, feature version, validation version, configuration path, MLflow run identifier, and cross-validated performance broken out by overall, masked, and unmasked regimes, along with its current status and recommendation. A separate submission log tracks every submission made to the competition platform, linked back to the candidate that produced it, recording the full three-tier cross-validated RMSE, the masked and unmasked decomposition, the cross-fold standard deviation, the degradation slope, the hypothesis the submission was testing, the resulting public leaderboard score where available, and a decision field — retain, discard, or investigate — together with a submission type classifying it as a sanity check, an architectural test, a model change, a feature change, an ensemble change, or a final candidate. This structure exists so the log shows what was learned from each submission, not merely an accumulating list of scores that nobody revisits, and it is treated as adequate at this project's scale — 200 total submissions, not thousands — without building a heavier automated registry service.

Out-of-fold predictions are stored as a first-class artifact rather than a transient notebook variable, because ensembling, residual correction, error-correlation-based final selection, and uncertainty estimation all silently produce wrong results if out-of-fold predictions are not reliably persisted and reused. Each candidate's out-of-fold predictions are stored per fold, with columns for location, forecast origin, target, prediction, and regime.

Each submission is accompanied by a prediction manifest — a serialization of its submission-log row together with a schema hash — so that every submitted file has a self-contained reproducibility record sitting alongside it, directly satisfying the competition's requirement to document data, outputs, features, package versions, and environment at the level of an individual submission, not only at the level of the project as a whole.

Given a budget of at most 200 total submissions and at most 5 per day, submissions are allocated roughly across the competition timeline: early submissions establish validation sanity checks against the platform's own scoring; mid-competition submissions test each meaningful step up the champion ladder live; late submissions are reserved for fine-tuning around the strongest candidates.

Near the competition's close, final selection of the two submissions nominated for private-leaderboard judging is made using error correlation rather than cross-validated score alone: for each shortlisted candidate, cross-validated performance with its variance, public leaderboard score, the regime-level decomposition, out-of-fold error correlation against the other shortlisted candidates, and a risk note describing dependence on external data or architectural complexity are all reported together, and the two candidates chosen are the two that are genuinely different from each other by error correlation, not simply the two with the lowest cross-validated RMSE. Two candidates with RMSE of 0.520 and 0.525 but an error correlation of 0.99 offer almost no diversification benefit against the unseen private 70% of the test set; two candidates with RMSE of 0.520 and 0.528 but an error correlation of 0.75 could meaningfully hedge that risk, and are preferred on that basis.

SECTION 15

Definition of champion

The champion is not defined as whichever model currently holds the best public leaderboard score. It is defined as the reproducible model or ensemble judged to have the strongest expected private-leaderboard performance under the most realistic available validation architecture, subject to the leakage, robustness, reproducibility, and competition-rule constraints described throughout this document. The public leaderboard is one piece of evidence toward that judgment; it is not the authority the judgment is made against. The current champion retains its position until a challenger clears the identical bar — validation performance, the masked-regime requirement, reproducibility, robustness, and complexity justified by evidence — which is a useful framing across a multi-week competition in which many candidates are expected to be tried and discarded.

The champion is also not automatically the model that is deployed in the interactive application. A research release — a model that is scientifically validated and reported on — a competition release — a model that is submission-ready for the leaderboard — and a production release — a model that has passed the deployment eligibility checks described in Project Phase 13 — are permitted to be different points in time, and in principle different candidates entirely, without that being treated as an inconsistency in the system.

SECTION 16

Assumptions and open questions

Two living registers record, respectively, what the system currently believes and acts on without yet having proven, and what the system currently does not know and does not expect to resolve soon. Both exist because this material is exactly the kind of thing that quietly disappears into working notebooks if it is not written down deliberately, and both are direct, honest inputs to the competition's Trustworthiness report.

Assumptions register

The assumptions register records six standing assumptions.

1. The test masking mechanism is believed, with high confidence, to represent the real GRACE-to-GRACE-FO observational gap rather than an artificial competition construct, based on the blackout months' timing matching documented GRACE mission history and the whole-month rather than per-row structure of the masking.
2. If external reanalysis or Copernicus data is sourced, it is assumed, with medium confidence, that it can genuinely be made available at the correct prediction-time vintage, though this has not yet been tested and depends on the specific source's revision policy.
3. Historical spatial relationships between neighboring grid cells are assumed, with medium confidence, to be stable enough across the 2002–2015 record to generalize into the 2016–2018 test period; same-month spatial correlation has been measured at 0.981 in-sample, but stability of that relationship over time specifically has not yet been directly tested.
4. The 2015 persistence-RMSE anomaly is assumed, with low confidence and under active investigation, not to represent a fundamental regime break that would invalidate 2002–2014 patterns for the test period — the anomaly itself is confirmed, but its cause is not yet determined, and this assumption is treated as the most fragile one currently in the system.

5. The public and private leaderboard split is assumed, with medium confidence, to be a random row-level sample rather than a structured or chronological one, consistent with Zindi's own stated language, though this is not independently verifiable without submission-probing, which is out of bounds.
6. It is assumed, with medium confidence, that a carefully engineered gradient-boosted pipeline built on the state-reconstruction architecture will match or beat deep spatiotemporal models on a dataset of this scale — roughly 2.15 million rows across 15,715 locations — based on general reasoning about data scale and tabular-competition norms, not yet on a direct empirical test specific to this problem.

Open-questions register

The open-questions register records five questions expected to remain unresolved for some time rather than to be settled quickly.

1. Why persistence RMSE spikes specifically to 0.898 in 2015, against a 0.50–0.63 range in every other year, remains open and is the system's current highest-priority next action, because it determines how much the 2002–2014 record can be trusted as a guide to the 2016–2018 test period.
2. How much of the masked regime is genuinely predictable from environmental forcing alone, independent of any TWS history, remains open; a linear reconstruction of concurrent TWS from covariates achieves only 0.806 to 0.821, barely above the global mean of 0.912, but this has only been tested in a linear form so far, and a nonlinear, gradient-boosted version of the same test is the planned next step.
3. How stable spatial relationships are across the full 2002–2015 record, rather than within any single month, remains open, and the medium-confidence spatial-stability assumption above depends directly on it.
4. Whether external hydrological data genuinely improves out-of-time performance, rather than only improving in-sample fit, remains open and untested, and is central to whether the effort and provenance-gate overhead of sourcing external data is worthwhile at all.

5. Whether the public and private leaderboard split is uniformly random or could be structured in some way that is not directly observable remains open and is likely to remain open permanently, since it is not independently testable within the competition's rules.

SECTION 17

Explicit anti-goals

A number of approaches are deliberately not pursued, stated explicitly so that they are recognized and rejected quickly if proposed again rather than re-litigated.

- Reverse-engineering the structure of the public and private leaderboard split through deliberate submission probing is out of bounds, distinct from the entirely normal and planned practice of using test rows' own covariates as model inputs.
- Chasing improvements to the public leaderboard score that are not matched by a corresponding improvement in internal, honest cross-validation is avoided.
- Per-location climatology or simple linear imputation are not treated as standalone answers to the masked-row problem, since both are directly contradicted by the measured evidence in §3.
- Fully independent models trained separately per location or per continent are not adopted as a first move, and are only considered at all if using regime as a model feature proves genuinely insufficient, given that most locations have only around 150 monthly observations to learn from.
- A Transformer- or graph-neural-network-based architecture is not built before a gradient-boosted baseline exists and has been measured.
- Training 15,715 separate per-location models is avoided in favor of the shrinkage-regularized signature approach described in §10.
- Multiple days are not spent on gradient-boosted hyperparameter tuning before the feature and state representation from Project Phase 4 is settled, since representation matters substantially more than tuning for this problem.

- Random, row-independent masking augmentation during training is avoided, since it does not reproduce the real contiguous blackout process described in §8 and would optimize the model against an easier synthetic problem than the one it will actually face.
- Data-versioning tooling, continuous integration, continuous deployment, and other production-engineering concerns are never allowed to block or slow down the modeling track, consistent with Project Phase 0's explicit split between what is required immediately and what can be added later.
- Manual, hand-adjustment editing of a generated prediction file is never performed, for any reason, because it breaks the reproducibility chain described in §18.
- Feature selection is never driven by which features happened to move the public leaderboard score, as distinct from features that moved honest cross-validated performance — the two are related but not the same signal, and only the second is trusted for selection decisions.

SECTION 18

Reproducibility

Four distinct kinds of reproducibility are recognized, because they have different audiences and different failure modes, and a claim of "the system is reproducible" is not meaningful until it specifies which kind is meant.

Scientific reproducibility

Asks whether someone else could reproduce the system's *conclusions* — for instance, that masked-regime state reconstruction outperforms naive imputation — which is supported by the assumptions and open-questions registers in §16 together with the experiment record maintained across the project.

Computational reproducibility

Asks whether someone could reproduce the system's *exact predictions*, bit for bit, given the same random seed, which is supported by centralized, deterministic seed control throughout the pipeline.

Software reproducibility

Asks whether someone could reinstall the system's environment and run it at all, which is supported by the pinned dependency specification and the exact resolved-environment capture described in §7.

Competition reproducibility

Asks specifically whether Zindi can re-run a submitted top-10 solution's own code and reproduce its own leaderboard score, which is supported directly by the per-submission prediction manifest described in §14, in combination with the environment capture from §7.

SECTION 19

The governing rule

Any final prediction produced by this system must be able to answer five questions.

1. Where did the underlying data come from — its lineage.
2. How was the hydrological state reconstructed — its feature and state lineage.
3. Why was this particular model selected over the alternatives that were tried — its experiment and validation lineage.
4. Why should the prediction be trusted — its validation performance, its explainability evidence, its robustness, and its provenance.
5. Can someone else reproduce it — through the combination of git history, configuration, dataset version, MLflow record, and a deterministic pipeline described throughout this document.

If any of these five questions cannot be answered for a given prediction, the system is not considered finished, regardless of its leaderboard score.

SECTION 20

Final architectural statement and current status

This system is a reproducible, regime-aware global water-storage forecasting pipeline, not a competition notebook that happens to produce a submission file. It:

- Reconstructs the hydrological state of each location from its historical observation record;
- Characterizes that state's temporal and spatial behavior;
- Simulates the real observation and masking process realistically rather than approximately;
- Validates every model honestly under conditions that actually resemble the ones it will be judged on;
- Models the evolution of that state using a progression of increasingly sophisticated, evidence-gated specialists;
- Combines complementary models through a disciplined out-of-fold methodology;
- Quantifies both bias and uncertainty explicitly;
- Registers a single, reproducible champion at any given time;
- Generates fully traceable competition submissions;
- Exposes the validated model through both a programmatic API and an interactive application a person can actually use.

The single design decision underlying all of it is not the choice of any specific model — it is the state-reconstruction-plus-process-aware-validation architecture described in §3 through §12. Every other component of the system exists to serve that architecture, and any future change to it is expected to be driven by actual experimental evidence, recorded through the Architectural Decision Record process described in §2, rather than by further speculative revision of this document.

As of this writing, the architecture described above is fully specified and the project's engineering foundation — repository structure, pinned and verified dependencies, and the data manifest — is in place. The 2015 persistence-RMSE anomaly described in §3 and §16 remains unresolved and is the system's next concrete action, followed immediately by the stratified blackout-degradation curve described in Project Phase 1 of §13. No modeling code has been written yet; the ordered experiment sequence in Project Phase 1 is where implementation begins.